

Red5-Server - Projet de Qualité Logicielle

1. Introduction
2. Modifications proprement dites
 - 2.1 petites modifications
 - 2.1.1 renommage de constante
 - 2.1.2 creation de variable pour supprimer un nombre magique
 - 2.1.3 simplification basiques de code
 - 2.1.3.1 simplification d'une concatenation
 - 2.1.3.2 simplification d'un return
 - 2.1.3.3 simplification d'un bloc de condition
 - 2.1.4 Suppression de code morts
 - 2.1.4.1 Suppression de if vide
 - 2.1.4.2 Suppression de variables locales stockées dans des methodes inutilisés
 - 2.1 moyennes modifications
 - 2.1.1 reduction de la complexite cognitive de playengine
 - 2.1.2 suppression de la duplication de codes entre les methodes
 - 2.1.3 ajout de test pertinent
 - 2.1.4 remplacer le fait qu'une methode retourne un code d'erreur par le fait qu'elle lève une exception
 - 2.1.5 corriger un test rouge en orange
 - 2.1.6 decomposer une methode qui à la fois retourne des information et modifie l'etat d'un objet
 - 2.2 grandes modifications
 - 2.2.1 ajout d'une super classes pour supprimer des methodes dupliquée
 - 2.2.2 decomposition d'une god classes
 - 2.2.3 fusion de classes
 - 2.2.4 suppression de classes static
 - 2.2.5 suppression de package contenant moins de classes
3. Bilan

Réalisé par :	Encadré par :
TCHINDE SINGHE RODERICK DARELL	Mme Clotilde Toullec

Dépôt Git du projet :

<https://github.com/roderick-darell/g>

—

Red5-Server - Projet de Qualité Logicielle



1. Introduction

après une première partie autour de l'évaluation de la qualité logicielle du projet **red5-server** avec mon binome ali hassanoui ;

il sera donc ici question dans cette deuxième partie et ceci de façon individuelle de proposer une série d'améliorations diverses et efficaces de ce logiciel d'envergure.

malgré le fait que mon binome et moi ayant travaillé en commun sur la première partie du projet et ayant donc cependant repéré les mêmes anomalies

logicielles, cette deuxième partie se faisant de façon individuelle toutes ressemblent avec les modifications de ces dernières ne seraient que fortuites .

dans la suite du projet toutes les modifications effectuées présentées se feront suivant le même canevas : identification de la problématique - modifications apportées - bilan de la modification et enfin un lien vers le commit concerné.

dans le but de faciliter les corrections, les modifications ont été classées en 3 grands groupes : petites, moyenne et grande .

2. Modifications proprement dites

2.1 petites modifications

2.1.1 renommage de constante

j'ai remarqué que la nomenclature de certaines constantes en java n'était pas respecté (TOUT EN MAJUSCULE).

À titre d'exemple j'ai renommé la variable **useragent** du fichier

common/src/main/java/org/red5/server/util/HttpConnectionUtil.java en **USER_AGENT**

cette modification constitue à l'amélioration de la syntaxe globale du projet à savoir le respect de la syntaxe java

adresse du commit: <https://github.com/roderick-darell/gl/commit/62031252f0049d3f1840204ff17080bb330b1d7f>

2.1.2 creation de variable pour supprimer un nombre magique

j'ai remarqué à plusieurs endroits du code cette portion de code `ioBuffer buf = ioBuffer.allocate(102)` entre autres dans le fichier `common/src/main/java/org/red5/server/stream/PlayEngine.java`; où 102 représentait la taille initiale allouée au buffer.

j'ai donc créé une variable `int INITIAL_BUFFER_SIZE = 102;`. ceci permet non seulement de rendre le code plus lisible mais surtout de faciliter sa compréhension car il

informe de la nature et de la présence des nombres précis dans le code.

adresse du commit correspondant: <https://github.com/roderick-darell/gl/commit/be9f873f4f749fba97e8cb6ad0818843ec8ed3c7>

2.1.3 simplification basiques de code

2.1.3.1 simplification d'une concaténation

dans le fichier `common/src/main/java/org/red5/server/ContextLoader.java` nous remarquons la concaténation suivante avec la chaîne vide `String configReplaced = config + ""`;

nous l'avons simplifié en supprimant cette concaténation.

le résultat ne change rien mais il permet entre autre de rendre notre code plus lisible.

adresse du commit: <https://github.com/roderick-darell/gl/commit/0c2c3754dd4bfba0ef60d1d07dc8b97bf768e462>

2.1.3.2 simplification d'un return

dans le fichier `common/src/main/java/org/red5/server/net/rtmp/message/Package.java` on souhaitait retourner vrai (message expire) si le temps d'expiration est égal à zéro.

j'ai donc remplacé `return expirationTime > 0L ? System.currentTimeMillis() > expirationTime : false;` par `return expirationTime > 0L && System.currentTimeMillis() > expirationTime;`

le but de cette modification est de rendre le code plus maintenable car il est plus facilement compréhensible.

adresse du commit: <https://github.com/roderick-darell/gl/commit/5494a2724b7ccd643ae51e403757561407f3fe3b>

2.1.3.3 simplification d'un bloc de condition

j'ai remarque dans le fichier common/src/main/java/org/red5/server/stream/PlayEngine.java on avait un bloc du genre if (condition) return false else return true

je l'ai simplifié en return !condition .

le but ici est juste de simplifier le code pour qu'il soit plus intepretable par un humain.

adresse du commit:<https://github.com/roderick-darell/gl/commit/7826d7fc60f225b237546caec6a58e61cc5e397a>

2.1.4 Suppression de code morts

2.1.4.1 Suppression de if vide

dans le fichier common/src/main/java/org/red5/server/stream/VideoFrameDropper.java je me suis rendu compte qu'il y'avait des if vides

nous avons donc supprimer ce elses inutiles. cela permet ainsi non seulement de reduire la complexite cognitive de la methode en question mais de

permettre une meilleur comprehension en vie d'une maintenace future.

adresse du commit:<https://github.com/roderick-darell/gl/commit/2f69308660d5001538ba5a26f6bfb4b091d68b94>

2.1.4.2 Suppression de variables loycales stockées dans des methodes inutilisés

dans le fichier common/src/main/java/org/red5/server/stream/PlayEngine.java je me rends compte que dans une methode on stocke le message dans une variable locale et on l'utilise pas donc a la sortie de la methodes la variable est detruite.

j'ai donc remplacer cette instruction (message = RTMPMessage.build(body)) par RTMPMessage.build(body); cela contribue a reduire la comlexite en espace de cette methode car on economise une variable cree gratuitement.

adresse du commit:<https://github.com/roderick-darell/gl/commit/7fb7d227a87afc84c16920d988f6c94abb5fd267>

2.1 moyennes modifications

2.1.1 reduction de la complexite cognitive de playengine

la methodes playengine avait été precedemmmment identifié comme une à la plus grande complexité cognitives .

je remarque au sein de celle ci la methodes executes et play qui ont les deux plus grandes complexite cognitive au sein de celle ci et meme intelli la soulig,ait comme etant problematiques

a coup d'extraction multiples de methodes au sein de chacune de ce deux methodes j'ai reusssi a reduire la complexite de cette classe de plus de 100 et ainsi a respecter

les standartd de notre ide qui ne les marquait plus comme problematique.

adresse du commit 1:<https://github.com/roderick-darell/gl/commit/395a6558eb7ad9f61aece5068c0a356bbe680b2e>

adresse du commit 2:<https://github.com/roderick-darell/gl/commit/f04c592adf32d63be64f6a15b1fafbbdf49fd1af>

ci desssous deux captures presentant la situationnavant modification et celle après

The screenshot displays three SonarLint issues in an IDE. Each issue is a 'Brain Method' warning for high Cognitive Complexity. The first issue is for the `pushMessage` method (180 to 15 allowed), the second for the `execute` method (71 to 15 allowed), and the third for a `switch` statement (73 to 15 allowed). Below the issues, a file list shows the following files and their line counts:

File	Lines
OBUParser.java	727
server/stream/PlayEngine.java	716
impl/MP4Reader.java	437
u/OBUParser.java	727
5/server/stream/PlayEngine.java	626
p4/impl/MP4Reader.java	437

2.1.2 suppression de la duplication de codes entre les methodes

2.1.3 ajout de test pertinent

!!! a lire après avoir lu la section [creation d'une sous classes pour supprimer la duplication](#)

Avant refactoring, plusieurs classes (AudioData, VideoData, Aggregate) contenaient une logique dupliquée pour l'initialisation des données, ce qui augmentait le risque d'incohérences et compliquait la maintenance. L'introduction de la superclasse BaseDataEvent a permis de centraliser cette logique commune, améliorant la lisibilité et la cohérence du code. Toutefois, ce déplacement de comportement critique pouvait introduire des régressions. Le test ajouté vise précisément à valider ce point de mutualisation en vérifiant les deux scénarios essentiels (copie et référence du buffer), garantissant ainsi que le refactoring n'a pas altéré le comportement attendu. Il joue donc un rôle clé en sécurisant la qualité et la fiabilité du code après modification. Suite à la mise en place de la superclasse BaseDataEvent pour réduire la duplication de code, les modifications ont été validées par inadvertance sans avoir été correctement ajoutées au préalable (git add). Cela a conduit à une séparation des changements sur plusieurs commits distincts, notamment entre l'introduction de la nouvelle abstraction, la suppression du code dupliqué et l'ajout des tests associés. Les trois commits référencés ci-dessous retracent ainsi l'évolution complète de ce refactoring : création de la superclasse, adaptation des classes existantes et ajout du test de validation. Cette situation n'impacte pas le résultat final mais reflète un enchaînement non optimal dans la gestion des commits.

adresse du commit 1: <https://github.com/roderick-darell/gl/commit/8652391035a4561aa22befe0cd6cfba3446a5164>

adresse du commit 2: <https://github.com/roderick-darell/gl/commit/1d04a60652f6cff754c9e1a535d5840d8faf7426>

adresse commit 3: <https://github.com/roderick-darell/gl/commit/33a83314b59685833bc7f678c8c03f4622a52f1b>

2.1.4 remplacer le fait qu'une methode retourne un code d'erreur par le fait qu'elle lève une exception

dans la classe common/src/main/java/org/red5/server/stream/PlayEngine.java il y'avait une methode qui retournait -1 lorsque le resultat de

oobCtrlMsg.getResult() n'etait pas un entier au lieu de cela j'ai cree une exception SendVODSeekCMException puis je l'ai lancer en lieu et place de retourner -1 .

l'avantage est que lorsque on tombera sur ce cas de figure on saura exactement ce qu'il l'a causé et on pourra meme faire un try catch pour la capturer et en faire bonne usage.

```
tchinde-singhe@tchinde-singhe-ASUS-Vivobook-15-X1504VAPF-X1504VA:~/s6/gl$ mvn -pl common -Dtest=TestBaseDataEvent test
[INFO] Scanning for projects...
[INFO]
[INFO] -----< org.red5:red5-server-common >-----
[INFO] Building Red5 :: Server Common 2.0.30
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- formatter-maven-plugin:2.19.0:format (default) @ red5-server-common ---
[INFO] Processed 282 files in 402ms (Formatted: 0, Unchanged: 282, Failed: 0, Readonly: 0)
[INFO]
[INFO] --- maven-resources-plugin:3.3.1:resources (default-resources) @ red5-server-common ---
[INFO] skip non existing resourceDirectory /home/tchinde-singhe/s6/gl/common/src/main/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.13.0:compile (default-compile) @ red5-server-common ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- maven-resources-plugin:3.3.1:testResources (default-testResources) @ red5-server-common ---
[INFO] Copying 2 resources from src/test/resources to target/test-classes
[INFO]
[INFO] --- maven-compiler-plugin:3.13.0:testCompile (default-testCompile) @ red5-server-common ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- maven-surefire-plugin:3.0.0:test (default-test) @ red5-server-common ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitcore.JUnit4CoreProvider
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[WARNING] Couldn't load group class 'org.red5.test.IntegrationTest' in Surefire\Failsafe plugin. The group class is ignored!
[INFO] Running org.red5.server.net.rtmp.event.TestBaseDataEvent
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.158 s - in org.red5.server.net.rtmp.event.TestBaseDataEvent
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 2.075 s
[INFO] Finished at: 2026-03-29T19:06:39+02:00
[INFO]
[INFO] -----
tchinde-singhe@tchinde-singhe-ASUS-Vivobook-15-X1504VAPF-X1504VA:~/s6/gl$
```

adresse du commit:<https://github.com/roderick-darell/gl/commit/ec9cc532d335fd3ae3ea7d7db399a72f23b35ee9>

adresse du commit de creation de la classe d'exception:<https://github.com/roderick-darell/gl/commit/d8169893544b01c0e6bf788dd52e796ffc456992>

2.1.5 corriger un test rouge en orange

Initialement, certains tests étaient commentés car leur exécution entraînait des erreurs, notamment un échec du test testGetStreamById. En l'état, le test ne passait pas car il utilisait un identifiant de type Integer alors que l'implémentation de getStreamById convertit systématiquement l'identifiant en Double pour accéder à la structure interne. Cette incohérence de type provoquait un retour null inattendu et donc l'échec de l'assertion. La correction a consisté à aligner le test sur le comportement réel de la méthode en utilisant une clé de type Double lors de l'insertion dans la map. Après cette adaptation, le test est redevenu cohérent avec l'implémentation et s'exécute désormais avec succès.


```

[INFO] -----
[ERROR] /home/tchinde-singhe/s6/gl/common/src/test/java/org/red5/server/net/rtmp/TestRTMPConnection.java:[68,9] cannot find symbol
symbol:   class IClientStream
location: class org.red5.server.net.rtmp.TestRTMPConnection
[INFO] 1 error
[INFO] -----
[INFO] BUILD FAILURE
[INFO] -----
[INFO] Total time: 3.228 s
[INFO] Finished at: 2026-04-03T10:05:04+02:00
[INFO] -----
[ERROR] Failed to execute goal org.apache.maven.plugins:maven-compiler-plugin:3.13.0:testCompile (default-testCompile) on project red5-server-common: Compilation failure
[ERROR] /home/tchinde-singhe/s6/gl/common/src/test/java/org/red5/server/net/rtmp/TestRTMPConnection.java:[68,9] cannot find symbol
[ERROR] symbol:   class IClientStream
[ERROR] location: class org.red5.server.net.rtmp.TestRTMPConnection
[ERROR]
[INFO] -----
[WARNING] Couldn't load group class 'org.red5.test.IntegrationTest' in Surefire\Failsafe plugin. The group class is ignored!
[INFO] Running org.red5.server.net.rtmp.TestRTMPConnection
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.276 s - in org.red5.server.net.rtmp.TestRTMPConnection
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 4.788 s
[INFO] Finished at: 2026-04-03T10:28:58+02:00
[INFO] -----
tchinde-singhe@tchinde-singhe-ASUS-Vivobook-15-X1504VAPF-X1504VA:~/s6/gl$

```

adresse du commit: <https://github.com/roderick-darell/gl/commit/ba47dea2ec7dac4708ecae5ee6f6d95f73665ee4>

2.1.6 decomposer une methode qui à la fois retourne des information et modifie l'etat d'un objet

dans le fichier common/src/main/java/org/red5/server/ClientRegistry.java ,Initialement, la méthode newClient combinait deux responsabilités distinctes : la création d'un client à partir des paramètres et son enregistrement dans la connexion via addClient. Cette double responsabilité rendait le code moins lisible et plus difficile à tester, car la logique de création était mêlée à un effet de bord. Le refactoring a consisté à séparer ces responsabilités en introduisant des méthodes dédiées (createClient pour la création et registerClient pour l'enregistrement), ainsi qu'une méthode auxiliaire pour extraire l'identifiant. Cette nouvelle organisation améliore la lisibilité, la modularité et la testabilité du code, tout en rendant les intentions de chaque méthode plus claires.

adresse du commit: <https://github.com/roderick-darell/gl/commit/b0c87da021c7193342ad834ebd51b84ce046c02e>

2.2 grandes modifications

2.2.1 ajout d'une super classes pour supprimer des methodes duppliée

je remarque que parmi les differents type de format destreamingb(classes filles de baseEvent) certains ont un pattern qui se repete dans leur constructeur ;

et etant donné que toutes n'etait pas concerné je ne pouvais pas faire monter la classe dans baseEvent j'ai donc creer une superclasse regroupant ces dont herite les codecs

video ,audio et aggregate et c'est dans celle ci que j'ai centralisé le pattern se repetant.

cela est interessant parceque au dela du pattern de construction ces trois classes partagent ensemble beaucoup d'autres duplication de codes ce qui suggère

que d'un point de vue metier il est interressant d'avoir une super classe pour elle.

!!! après avoir lu ceci bien vouloir lire directement la section xxx

2.2.2 decomposition d'une god classes

Initialement, la classe PlayEngine présentait les caractéristiques d'une god class, concentrant de nombreuses responsabilités telles que la gestion des notifications, l'envoi de messages de contrôle, la gestion des tâches planifiées et la surveillance du buffer client. Cette accumulation rendait le code difficile à lire, à maintenir et à tester. Afin d'améliorer sa qualité, une décomposition progressive a été réalisée en extrayant plusieurs classes spécialisées (PlaybackNotifier, PlaybackControlSender, PlaybackJobController, PlaybackDecisionResolver, ClientBufferMonitor). Cette refactorisation permet de mieux séparer les responsabilités, de réduire le couplage et de clarifier le rôle de chaque composant. Le PlayEngine devient ainsi un orchestrateur plus lisible, tandis que les nouvelles classes sont plus simples, réutilisables et testables individuellement.

io/src/main/java/org/red5/io/obu/OBUParser.java	727
common/src/main/java/org/red5/server/stream/PlayEngine.java	716
io/src/main/java/org/red5/io/mp4/impl/MP4Reader.java	437
io/src/main/java/org/red5/io/obu/OBUParser.java	439
common/src/main/java/org/red5/server/stream/PlaybackControlSender.java	361
common/src/main/java/org/red5/server/net/rtp/RTMPCConnection.java	300

apres

Cognitive Complexity 13,930	New Code: Since March 4, 2026
io/src/main/java/org/red5/io/obu/OBUParser.java	727
common/src/main/java/org/red5/server/stream/PlayEngine.java	583
io/src/main/java/org/red5/io/mp4/impl/MP4Reader.java	437

Cyclomatic Complexity 12,597	New Code: Since March 4, 2026
io/src/main/java/org/red5/io/obu/OBUParser.java	439
common/src/main/java/org/red5/server/stream/PlayEngine.java	357
common/src/main/java/org/red5/server/net/rtp/RTMPCConnection.java	300

adresse des commit 1:<https://github.com/roderick-darell/gl/commit/45fd3277a31e656224084c88dd5a30c4bd9a1a09>

adresse des commit 2:<https://github.com/roderick-darell/gl/commit/031f034c6b1694a9cfadb003ad64dbb420ae5c90>

adresse des commit 3:<https://github.com/roderick-darell/gl/commit/d74879cd51560158df70adaf0bb11efa9871912a>

adresse des commit 4:<https://github.com/roderick-darell/gl/commit/482b694f74b1083310c5fe58c893fc83bcc22413>

adresse des commit final :<https://github.com/roderick-darell/gl/commit/29a8f0c6e5c15355fc875e366a05560d1124a408>

2.2.3 fusion de classes

Dans le fichier de gestion des flux, une duplication importante a été identifiée entre les classes SingleItemSubscriberStream et PlaylistSubscriberStream. L'objectif initial était de réduire cette duplication. Après analyse du code, il est apparu que ces deux classes partageaient une grande partie de leur logique (gestion du moteur de lecture, scheduling, traitement des événements, etc.), avec pour principale différence que l'une manipulait un seul élément tandis que l'autre gèrait une liste. D'un point de vue métier, cette distinction est artificielle : un flux unique peut être vu comme une playlist contenant un seul élément.

Afin d'unifier ces comportements, la classe `PlaylistSubscriberStream` a été adaptée pour supporter ce cas particulier via l'ajout d'une méthode `setPlayItem(IPlayItem item)`, permettant de simuler un mode "single" en utilisant une playlist de taille 1. Une fois cette compatibilité mise en place, la classe `SingleItemSubscriberStream` a été supprimée à l'aide de la fonctionnalité Safe Delete de IntelliJ, ce qui a permis d'identifier automatiquement tous ses usages dans le projet. Ces usages ont ensuite été progressivement remplacés par `PlaylistSubscriberStream`, en conservant les appels existants comme `setPlayItem(...)`.

Cette refactorisation améliore la qualité du code à plusieurs niveaux. Elle supprime la duplication de logique, réduit le nombre de classes à maintenir et simplifie le modèle métier en unifiant deux concepts fortement liés. Elle facilite également les évolutions futures, puisque toute modification du comportement de lecture est désormais centralisée dans une seule classe. Enfin, elle améliore la lisibilité et la cohérence du code en alignant l'implémentation sur une vision métier plus juste : une playlist est un concept générique capable de représenter aussi bien un flux multiple qu'un flux unique.

adresse du commit : <https://github.com/roderick-darell/gl/commit/1d1f46ca9aaed8a95c4f04577390983d4c4d67db>

2.2.4 suppression de classes static

Dans l'état initial du projet, la classe `HttpConnectionUtil` était conçue comme une classe utilitaire statique. Elle exposait plusieurs méthodes statiques (`getClient`, `getSecureClient`, `handleError`) tout en maintenant un état global mutable à travers des attributs statiques tels que `connectionManager` et `connectionTimeout`. Cette conception introduisait plusieurs problèmes : un fort couplage entre les différentes parties du code, une difficulté à contrôler et tester le comportement, ainsi qu'un risque de comportements imprévisibles liés au partage d'état global entre plusieurs composants ou threads.

La problématique principale résidait donc dans le fait que cette classe combinait des responsabilités de type "utilitaire" avec une logique de configuration et de gestion d'état, ce qui va à l'encontre des principes de conception orientée objet. De plus, toute modification du timeout ou du gestionnaire de connexions affectait globalement l'ensemble de l'application, ce qui pouvait entraîner des effets de bord difficiles à diagnostiquer.

Pour remédier à cela, la classe a été refactorisée en une classe instanciable. Les attributs statiques ont été transformés en attributs d'instance, le bloc d'initialisation statique a été remplacé par un constructeur, et les méthodes ont été rendues non statiques. Les appels existants dans le projet ont ensuite été adaptés pour utiliser une instance de cette classe plutôt que des appels statiques.

Cette nouvelle version présente plusieurs avantages. Elle améliore la testabilité en permettant d'instancier et de configurer indépendamment les objets selon le contexte. Elle réduit le couplage en supprimant la dépendance à un état global partagé. Elle rend également le comportement plus prévisible, notamment dans un environnement multi-thread comme celui du serveur Red5, où l'utilisation d'un état statique mutable pouvait entraîner des conflits ou des incohérences.

adresse du commit : <https://github.com/roderick-darell/gl/commit/f3214f7d68b8b15662aff47488c0533c6c4bc484>

2.2.5 suppression de package contenant moins de classes

Initialement, le package `org.red5.server.net.rtmpt.codec` ne contenait qu'un très petit nombre de classes fortement liées au fonctionnement de rtmpt, à savoir `RTMPTCodecFactory`, `RTMPTProtocolDecoder` et `RTMPTProtocolEncoder`. Cette séparation introduisait un niveau de package supplémentaire peu justifié, qui complexifiait inutilement l'arborescence et la navigation dans le code. La modification a consisté à remonter ces classes directement dans le package `org.red5.server.net.rtmpt`, puis à supprimer le sous-package `codec`.

Cette réorganisation est préférable car elle simplifie la structure du module, améliore la lisibilité et rapproche des classes qui collaborent étroitement dans un même espace fonctionnel. Elle réduit également la dispersion artificielle du code sans modifier le comportement métier, ce qui rend l'ensemble plus cohérent et plus facile à maintenir

adresse du commit: <https://github.com/roderick-darell/gl/commit/2f7664cf38a25db30127316d5605411e0fe89a2c>

3. Bilan

Le travail réalisé sur un projet de grande envergure comme Red5 m'a permis de mieux comprendre les enjeux concrets de la qualité logicielle. J'ai pu constater que, même au sein d'un projet maintenu par une équipe expérimentée, le code peut contenir des imperfections : tests commentés pour éviter des échecs, imports inutilisés, duplication de code ou encore structures parfois complexes et difficiles à maintenir. Cela montre que la qualité du code n'est jamais acquise et qu'elle nécessite une attention constante.

Cette expérience m'a également fait prendre conscience de l'importance du génie logiciel et des bonnes pratiques (tests, refactoring, structuration du code). Elles ne sont pas uniquement théoriques, mais essentielles pour garantir la maintenabilité, la lisibilité et l'évolutivité d'un projet réel.

Enfin, j'ai compris que chaque développeur a un rôle à jouer dans l'amélioration continue du code. La qualité logicielle ne dépend pas seulement d'outils ou de processus, mais aussi d'une démarche individuelle : être rigoureux, remettre en question l'existant et proposer des améliorations. C'est en adoptant cette posture que l'on contribue réellement à la qualité globale d'un projet.